Лабораторная работа 3. Улучшение поиска в RAG: BM25, объединение выдач и переранжирование

Главный вопрос. Какие ошибки плотного поиска исправляют лексический поиск BM25, объединение выдач и переранжирование — и доходит ли улучшение поиска до ответа модели?
Что получится. Четыре варианта поиска для системы ЛР2: плотный, BM25, объединение выдач (RRF) и RRF с переранжированием cross-encoder. Сравнение на dev, выбор варианта по заранее заданному правилу и однократная проверка на отложенной части; разбор трейсов; отчёт.
Среда. Система, данные и разбиение ЛР2; реализация BM25 (своя или библиотечная), cross-encoder из sentence-transformers; поиск — на CPU, cross-encoder быстрее на GPU.

1. Зачем эта работа

Разбирая ошибки ЛР2, вы, скорее всего, видели утверждения, для которых плотный поиск нашёл статьи на ту же тему, но не ту, где есть ответ. Возьмите утверждение «ALDH1 expression is associated with better breast cancer outcomes». В корпусе есть статья с заголовком «ALDH1 is a marker of normal and malignant human mammary stem cells and a predictor of poor clinical outcome». Для векторного поиска «ALDH1» — один из многих токенов, а по смыслу рядом окажутся десятки статей о раке молочной железы. Для лексического поиска это редкое слово, которое совпадает точно, и именно оно решает дело. В обратных случаях, когда в статье те же мысли выражены другими словами, лучше справляется плотный поиск.

Отсюда три идеи, которые вы проверите:

- **BM25** — классический лексический поиск: документ тем выше, чем больше в нём редких слов запроса [2];
- **объединение выдач** (Reciprocal Rank Fusion, RRF) — сложить два списка по позициям документов, чтобы документ, найденный обоими методами, поднялся выше [3];
- **переранжирование cross-encoder** — модель читает утверждение и документ вместе и оценивает их соответствие точнее, чем сравнение двух независимых векторов, но дороже: по одному прогону на каждую пару [4].

На SciFact такие приёмы работают неплохо: в сравнении BEIR лексический BM25 на этом наборе — сильный соперник плотных моделей [5]. Но рост метрики поиска ещё не означает, что модель станет отвечать лучше. И есть вторая ловушка, знакомая по ЛР0: если выбирать лучший вариант по тем же 90 утверждениям, на которых его сравнивали, оценка окажется оптимистичной. Поэтому вариант выбирается по dev по правилу, записанному заранее, а проверяется один раз на отложенной части.

2. Как устроено

2.1. BM25

BM25 развивает идею TF-IDF: слово запроса тем важнее для документа, чем чаще оно встречается в этом документе и чем реже — во всём корпусе. Как устроены TF-IDF и инвертированный индекс, благодаря которому такой поиск работает быстро, объясняют главы 1 и 6 учебника [1]. BM25 оценивает документ d для запроса q суммой по различным словам запроса:

$$s_{\mathrm{BM25}}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\,\frac{|d|}{\overline{\ell}}\right)}, \qquad \mathrm{IDF}(t) = \ln\!\left(1 + \frac{N - n_t + 0{,}5}{n_t + 0{,}5}\right).$$

Здесь $f(t,d)$ — сколько раз слово встречается в документе, $|d|$ — длина документа в словах, $\overline{\ell}$ — средняя длина по корпусу, N — число документов, n_t — число документов со словом t. Редкое

слово получает большой вес IDF; повтор слова в документе даёт убывающую прибавку (её регулирует k_1); длинный документ штрафуется (это регулирует b). В работе $k_1 = 1{,}2$ и $b = 0{,}75$, IDF — вариант Lucene [6].

Реализации BM25 различаются токенизацией и формулой IDF, поэтому название библиотеки не описывает метод. Токенизация в работе простая: нижний регистр и последовательности латинских букв и цифр, без стемминга и стоп-слов. `ALDH1` остаётся одним токеном `aldh1`, а `IL-6` превращается в `il` и `6`. Готовую библиотеку, например `bm25s` или `rank_bm25`, использовать можно, если её формула и параметры совпадают с описанными, — проверьте это на маленьком примере.

2.2. Объединение выдач (RRF)

Оценки BM25 и косинусные близости измеряются в разных шкалах, и просто сложить их нельзя. RRF складывает не оценки, а позиции [3]:

$$s_{\mathrm{RRF}}(d) = \sum_{m \in \{\text{dense, BM25}\}} \frac{1}{60 + r_m(d)}.$$

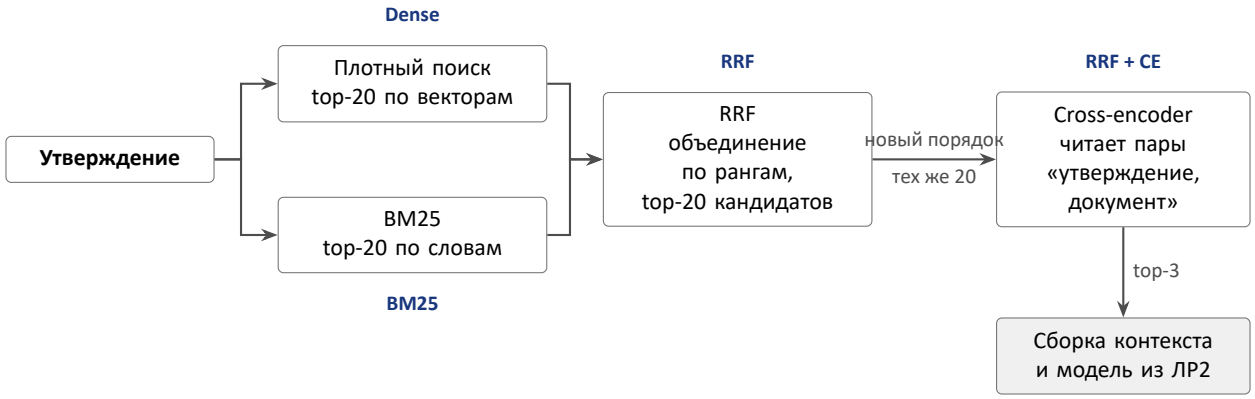
Здесь $r_m(d)$ — ранг документа в первых 20 результатах метода m. Если документа в списке метода нет, слагаемое равно нулю. Документ с рангами 1 и 4 получает $1/61 + 1/64$ и обходит документ, который был первым только у одного метода ($1/61$). Так RRF поощряет согласие методов, но может и отодвинуть полезную находку, которую сделал только один из них.

2.3. Переранжирование

Плотный поиск кодирует утверждение и документ по отдельности (bi-encoder). **Cross-encoder** получает их вместе как одну пару и выдаёт одну оценку соответствия: модель видит, как слова утверждения связаны со словами документа [4]. Заранее сохранить такие оценки нельзя — их считают для каждой пары заново. Поэтому cross-encoder применяют вторым этапом, к небольшому числу кандидатов.

В работе — `cross-encoder/ms-marco-MiniLM-L6-v2` [7]: он обучен на поисковых запросах MS MARCO, и насколько он переносится на научные утверждения, покажет опыт. Он переставляет 20 кандидатов RRF и новых документов не добавляет. Отсюда граница, которую полезно помнить: Recall@3 после переранжирования не может превысить Recall@20 кандидатов.

Рис. 1 показывает четыре варианта.



Стрелки показывают путь варианта RRF + CE. В вариантах Dense, BM25 и RRF модель получает первые три документа соответствующего списка. Reranker не добавляет новых документов: он только переставляет 20 кандидатов RRF.

Рис. 1. Четыре варианта поиска и что получает модель

2.4. Метрики и правило выбора

Метрики поиска — из ЛР2: Hit@k, Recall@k, MRR@k на утверждениях, для которых в разметке есть документы-свидетельства, то есть с метками `SUPPORT` и `CONTRADICT` (множество Q^+ из ЛР2, п. 2.6). Смотрим две глубины: $k = 3$ — столько документов получает модель, и $k = 20$ — запас кандидатов. Метрики ответа — Accuracy, полнота по классам, доля верных ссылок и доля переданного обоснования, как в ЛР2.

Правило выбора фиксируется до опытов: из вариантов BM25, RRF и RRF + CE выбирается тот, у которого на dev наибольший MRR@3; при равенстве — наибольший Recall@3, затем более простой вариант. Выбранный вариант сравнивается с плотным поиском на отложенной части. Итоговая рекомендация может оставить плотный поиск: выбор нужен, чтобы честно проверить лучшую альтернативу, а не чтобы обязательно её внедрить.

3. Что сделать

3.1. Данные и общие условия

Используйте корпус, разбиение (dev — 90 утверждений, holdout — 210) и манифест ЛР2; новое разбиение не создавайте. В Q^+ dev — 60 утверждений, holdout — 128. Модель, промпты, JSON Schema, бюджет контекста (2048 токенов) и $k = 3$ для генерации — как в ЛР2. Меняется только то, какие три документа получает сборщик контекста.

3.2. Четыре варианта

1. **Dense** — индекс ЛР2, но теперь сохраните первые 20 документов, а не 5. Проверьте, что первые пять совпадают с сохранёнными в ЛР2.
2. **BM25** — индекс по полному корпусу (заголовок и аннотация), токенизация и параметры из п. 2.1. Возвращаются до 20 документов с положительной оценкой; если совпадений нет, список пуст.
3. **RRF** — объединение двух списков по формуле из п. 2.2; первые 20 результатов.
4. **RRF + CE** — cross-encoder оценивает 20 кандидатов RRF: утверждение — первый текст пары, заголовок и аннотация — второй; предел 512 токенов, обрезается только второй текст. Список сортируется по оценке.

Во всех списках при равных оценках первым идёт меньший `doc_id`. Для каждого утверждения сохраните все четыре списка с оценками и рангами, а для RRF — вклады обоих методов.

3.3. Проверки перед опытами

- BM25 на корпусе из трёх коротких документов считается вручную и совпадает с вашей реализацией, включая слово, которое встречается в документе дважды, и запрос без совпадений.
- RRF на игрушечном примере: документ из обоих списков обходит документ, который был первым только в одном.
- После переранжирования множество 20 кандидатов не меняется, и Recall@20 остаётся прежним.
- Первые пять результатов Dense совпадают с ЛР2.

4. Эксперименты

До запусков запишите прогноз: для каких утверждений BM25 должен выиграть у плотного поиска; поможет ли RRF; изменит ли переранжирование ответы модели или только метрики поиска. Закоммитьте прогноз вместе с правилом выбора из п. 2.4.

4.1. Эксперимент 1. Дополняют ли методы друг друга

На 60 утверждениях Q^+ из dev сравните Dense, BM25 и RRF по Hit@3, Recall@3, MRR@3 и Recall@20. Затем составьте таблицу **дополнительности**: у скольких утверждений обоснование нашли оба

метода, только Dense, только BM25, ни один — на глубине 3 и на глубине 20. Для каждой группы посмотрите, сохранил ли RRF находку в первой тройке.

4.2. Эксперимент 2. Что даёт переранжирование

1. Сравните RRF и RRF + CE по тем же метрикам поиска на dev.
2. Выполните генерацию на 90 утверждениях dev для RRF и RRF + CE — это 180 вызовов. Ответы Dense с $k = 3$ возьмите из ЛР2, если модель, промпты и бюджет не менялись; иначе запустите Dense заново.
3. Сравните три варианта по Accuracy, доле верных ссылок и доле переданного обоснования; попарно — сколько ответов RRF + CE исправил и испортил относительно Dense.
4. Измерьте время поиска на запрос для каждого варианта отдельно: кодирование запроса и поиск, BM25, RRF, переранжирование. Медианы и диапазоны — по 90 запросам («Правила», п. 7).

4.3. Эксперимент 3. Выбор и отложенная проверка

1. Примените правило выбора из п. 2.4 к результатам dev. До запуска на holdout сохраните `selection.json`: выбранный вариант, значения метрик dev, коммит и хеш конфигурации.
2. Один раз запустите Dense и выбранный вариант на всех 210 утверждениях holdout: метрики поиска на 128 утверждениях Q^+ и генерация на всех 210. Это 420 вызовов модели.
3. Сравните результаты с dev. Если на holdout преимущество уменьшилось, это ожидаемо (ЛР0, эксперимент 3). Составы dev и holdout различаются — например, `CONTRADICT` в holdout заметно меньше, — поэтому показывайте результаты и по классам.

После просмотра результатов holdout параметры не меняются («Правила», п. 5). Если нашлась ошибка в коде, сохраните исходный результат, исправьте ошибку и обозначьте новый запуск как повторный.

4.4. Разбор трейсов

Разберите четыре случая: BM25 нашёл то, что пропустил плотный поиск; наоборот; RRF изменил первую тройку; переранжирование изменило ответ модели. Для каждого покажите путь: ранги в исходных списках → вклады RRF → порядок после переранжирования → переданные предложения → ответ. Назовите этап, на котором появилось или потерялось обоснование.

4.5. По желанию

- Подберите k_1, b или константу RRF на dev по небольшой сетке. Насколько вырос MRR@3 на dev и что стало на holdout? Это та же история, что с лучшим trial в ЛР0.
- Переформулируйте запрос моделью перед поиском (query rewriting) и сравните с исходным утверждением.
- Замените cross-encoder на модель, обученную на биомедицинских текстах.

5. Что сдать

- Код четырёх вариантов, конфигурации, `selection.json`; манифест (разбиение из ЛР2 и паспорта индексов).
- Артефакты: списки всех вариантов для dev и holdout, результаты генерации.
- Отчёт на 3–4 страницы: прогноз; таблица поиска на dev и таблица дополнительности; сравнение ответов на dev с попарными переходами; время поиска; итог на holdout рядом с dev; четыре разобранных случая; выводы.

Отсутствие улучшения — допустимый результат. Ошибки постановки здесь чаще всего такие: выбор варианта или параметров после просмотра holdout; сравнение ответов при разных бюджетах контекста; время чтения сохранённой выдачи вместо времени реального поиска.

Вопросы к защите

1. Покажите утверждение, где BM25 и плотный поиск дали разные первые тройки. Что в тексте утверждения и в токенизации это объясняет?
2. Документ на втором месте у Dense и отсутствует в выдаче BM25. Каков его вклад RRF? Что изменится, если ошибочно считать его рангом у BM25 двадцать первое место?
3. Почему Recall@3 после переранжирования не может превысить Recall@20 кандидатов? Может ли при неизменном Recall@20 измениться Accuracy?
4. MRR@3 вырос, а Accuracy не изменилась. Какие артефакты покажут, почему?
5. Сколько пар оценивает cross-encoder на всех запросах dev и holdout? Как это соотносится с временем поиска?
6. На holdout преимущество выбранного варианта меньше, чем на dev. Назовите две разные причины.
7. Замените константу 60 в RRF на 1. Предскажите, как изменится порядок на своём примере из вопроса 2, и пересчитайте без повторных вызовов модели.

Источники

1. Manning C. D., Raghavan P., Schütze H. Introduction to Information Retrieval. Cambridge University Press, 2008. Онлайн-версия — инвертированный индекс и TF-IDF (гл. 1, 6), BM25 (гл. 11.4.3), метрики поиска (гл. 8).
2. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 2009. Статья — вывод формулы BM25 и смысл параметров k_1 и b (разд. 3).
3. Cormack G. V., Clarke C. L. A., Buettcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR, 2009. Статья.
4. Nogueira R., Cho K. Passage Re-ranking with BERT. 2019. Статья — переранжирование cross-encoder вторым этапом поиска; практическое введение: Retrieve & Re-Rank в Sentence Transformers.
5. Thakur N. et al. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. NeurIPS Datasets and Benchmarks, 2021. Статья — сравнение BM25, плотного поиска и переранжирования на 18 наборах, включая SciFact (табл. 2).
6. Apache Lucene. BM25Similarity — вариант IDF и параметры по умолчанию.
7. Карточка cross-encoder/ms-marco-MiniLM-L6-v2.

Приложение. Детали реализации

Токенизация BM25: `re.findall(r"[a-z0-9]+", text.lower())`. Повтор слова в запросе второй раз не учитывается: сумма идёт по различным словам. Оценки считаются в `float64`.

RRF:

```
def rrf(lists, depth=20, c=60, top=20):
    scores = {}
    for ranked in lists:                          # каждый список — doc_id по убыванию оценки
        for rank, doc_id in enumerate(ranked[:depth], start=1):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (c + rank)
    return sorted(scores, key=lambda d: (-scores[d], int(d)))[:top]
```

Cross-encoder через Transformers:

```
from transformers import AutoModelForSequenceClassification, AutoTokenizer

name = "cross-encoder/ms-marco-MiniLM-L6-v2"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification.from_pretrained(name).eval()
batch = tok([claim] * len(docs), docs, truncation="only_second", max_length=512,
            padding=True, return_tensors="pt")
```

```
scores = model(**batch).logits.squeeze(-1)     # чем больше, тем выше документ
```

Объём вычислений. Cross-encoder оценивает 20 пар на запрос: 1800 пар на dev и 4200 на holdout. На CPU это минуты, на GPU — секунды. Генерация — 180 вызовов на dev и 420 на holdout; на T4 с моделью на 3–4 млрд параметров это 20–40 минут.